

Question 1 of 21

In the console, what does `console.log("2" + 3)` print?

☐

5

☐

An error, because you cannot mix text and numbers

☐

"23"

☐

6

Question 2 of 21

What is the piece inside every browser that actually reads and executes your JavaScript?

☐

The CSS stylesheet parser

☐

The browser's built-in JavaScript engine

☐

A server you must connect to

☐

The HTML validator

Question 3 of 21

In the console, what does `console.log(2 + 3)` print?

☐

"23"

☐

"5"

☐

An error

☐

5

What does `console.log("Aarav".length)` print?

☐

5

☐

"Aarav"

☐

4

☐

Aarav5

In a `switch` statement, what is the role of the `default` case?

☐ It runs when no `case` matches the value – like the `else` of a switch

☐ It always runs first, before any `case` is checked

☐ It sets the value that the switch compares each case against

☐ It is required in every switch, or the code throws an error

Question 7 of 21

A loop over 1..6 runs `if (i === 3) continue;` before printing `i`. What gets printed?

☐ 1, 2, 4, 5, 6 – the turn where `i` is 3 is skipped

☐ 1, 2 – the loop stops when `i` reaches 3

☐ 3, 4, 5, 6 – it jumps ahead to 3

☐ 1, 2, 3, 4, 5, 6 – `continue` has no effect

Question 8 of 21

Given `if (score >= 90) {...} else if (score >= 75) {...} else {...}` with `score = 82`, which block runs?

☐ The `else if (score >= 75)` block, because it is the first condition that is true

☐ Both the `else if` and the `else` blocks, since 82 qualifies for each

☐ The final `else` block, because 82 is not 90 or above

☐ None of the blocks, because 82 sits between the listed numbers

Question 9 of 21

What does `queue.shift()` do when `queue` is `["x", "y", "z"]`?

- ☐ Adds `"x"` to the start of the array
- ☐ Removes the LAST item, `"z"`
- ☐ Leaves the array unchanged and returns a copy
- ☐ Removes and returns the FIRST item, `"x"`, leaving `["y", "z"]`

Question 10 of 21

In `for (const t of tasks) { ... }`, what does `t` hold on each turn?

- ☐ Each VALUE of the array, one at a time, in order
- ☐ The index number (0, 1, 2, ...) of each item
- ☐ The entire `tasks` array on every turn
- ☐ Nothing — `for...of` requires a manual index

 Mark for review

Question 11 of 21

What does `stack.pop()` do when `stack` is `["a", "b", "c"]`?

- ☐ Removes and returns the FIRST item, `"a"`
- ☐ Removes and returns the LAST item, `"c"`, leaving `["a", "b"]`
- ☐ Adds a new item to the end of the array
- ☐ Returns the array's length

Question 12 of 21

For an array with 5 items, what is the index of the LAST item?

- ☐ 5, matching the number of items
- ☐ 4, because indexes run 0 through 4
- ☐ 1, because the last item is first from the end
- ☐ 6, one more than the count

Question 13 of 21

Which is a valid way to read the `price` property using BRACKET notation?

- ☐ `p["price"]`
- ☐ `p{"price"}`
- ☐ `p(price)`
- ☐ `p["price"]`

Question 14 of 21

Which line correctly creates an object with a `name` and a `price` property?

- ☐

```
const p = { name: "Mouse", price: 799 };
```
- ☐

```
const p = [ name: "Mouse", price: 799 ];
```
- ☐

```
const p = ( name = "Mouse", price = 799 );
```
- ☐

```
const p = < name: "Mouse", price: 799 >;
```

Question 15 of 21

`product.price` is already `799`. What does `product.price = 699;` do?

☐ Adds a second, separate `price` property

☐ Throws an error because `price` already exists

☐ Creates a brand-new object

☐ Overwrites the existing value, so `price` is now `699`

[< Previous](#)

Question 16 of 21

Given `const user = { name: "Aarav", greet() { return this.name; } };`, what does `user.greet()` return?

☐ `"Aarav"`

☐ `undefined`

☐ `"name"`

☐ The literal text `"this.name"`

Question 17 of 21

Which is a valid arrow function that takes exactly one parameter?

☐ `n, => n * 2`

☐ `(n => n * 2`

☐ `n => n * 2`

☐ `=> n (n * 2)`

Question 18 of 21

Inside a template literal, how do you insert the value of a variable named `name`?

☐ `{name}`

☐ `${name}`

☐ `%name%`

☐ `<name>`

Question 19 of 21

In `const square = n => n * n;`, why is there no `return` keyword?

☐ Because arrow functions can never return a value

☐ A single-expression arrow body returns its value automatically (implicit return)

☐ Because `n * n` is being stored back into `n`

☐ Because `return` is illegal inside any arrow function

[< Previous](#)

Question 20 of 21

After `const { price } = { name: "Pen", price: 40 };`, what is `price`?

☐ `"Pen"`

☐ `40`, matched by the key name

☐ The whole object

☐ `undefined`

[Clear Response](#)

Problem Statement

Implement **10 JavaScript functions** for a small **Task Tracker** – the data layer behind a to-do app. You write **no HTML**: all your code goes in

`src/app.js`.

A **task** is a plain object shaped like:

```
{ title: "Buy milk", status: "todo", done: false, priority: "high" }
```

`index.html` is a **Jasmine Spec Runner** – open it in a browser and it automatically runs the 10 tests against your functions. All 10 start **red**, make every one **green**.